GCA Session 4 - Lab Report

Maxime Rombaut
Master of Electronics and ICT Engineering Technology

Abstract: In this report, we present our work on developing a CUDA-based application that has 2 components. The first component takes an input image and converts it to its grayscale version using different memory accesses. The second component multiplies two matrices using different methods. Finally a comparison is made.
Keywords: CUDA, GPU, CPU, C++

1 Coalesced vs uncoalesced memory access

We read the pixel values of an input image and place those in a 1-D array so the pixel values will be as followed: RGBRGB...RGB. We want to convert it to its grayscale image and do this in two ways. The first way is uncoalesced, the second coalesced.
In the uncoalesced implementation, we reformat the input array so the pixel values have shifted positions. The array will look as followed: RRR...RGGG...GBBB...B. What we wanted to try and show with this is that the threads within the same warp would have to access different parts of the memory. This would create some latency.
In the coalesced implementation, the threads in the same warp would access parts of the memory that are adjacent to each other. This would have been more efficient than the uncoalesced access because the GPU can retrieve all the required data in one transaction.

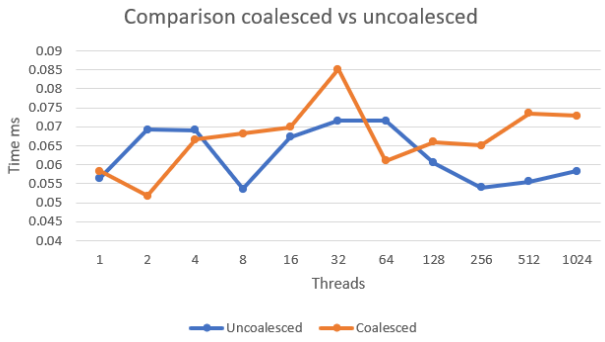


Figure 1: *coalesced vs uncoalesced memory access comparison*

As we can see in the graph, there isn't much difference between the two implementations. This is because all the memory has to be read out anyway. If we would only change the red pixel value, we would see different results. The RGBRGB implementation would be slower because the R values are further apart and there would be more memory transactions needed than in the other implementation.

2 Matrix multiplication

In this part of the application we want to compare the different types of memory: global, shared and constant.

2.1 Global memory

In the first implementation we only use global memory. All threads have access to this memory but it has high latency.

2.2 Shared memory

The shared memory is rather small. Because of this, not all threads can access it at the same time. We need to use striding to shift the threads in such a way that every calculation is done correctly. As in the global memory, we copy our matrices to the global memory. We then copy some of the matrix elements to the shared memory. These are elements of both matrices that will need to be multiplied. It would not be useful to put values of those matrices in the shared memory that don't need to be multiplied with each other. We then synchronize all threads and start multiplying. This is repeated until every value of the resulting matrix is calculated.

2.3 Constant memory

Constant memory can, like global memory, be access by all threads. It is optimized for low-latency reading. Because of this, the implementation is mostly the same as with the global memory. The only pitfall of this memory is that it is rather small as will become clear in the comparison.

2.4 Comparison

The different implementations have been tested on different matrix sizes. The sizes displayed on the graph are the sizes of one row in the matrix. So the total matrix would be size x size. The global and shared memory have been tested with 16 and 32 as block sizes. The CPU is the fastest for small matrices. For bigger matrices it is much slower than the others. For 2048x2048 matrices, it took to long so we interrupted the measurement.
Because the constant memory is rather small, we couldn't use it for matrices bigger than 64x64. A thing we could have tried was to fill the constant memory and then copy the other

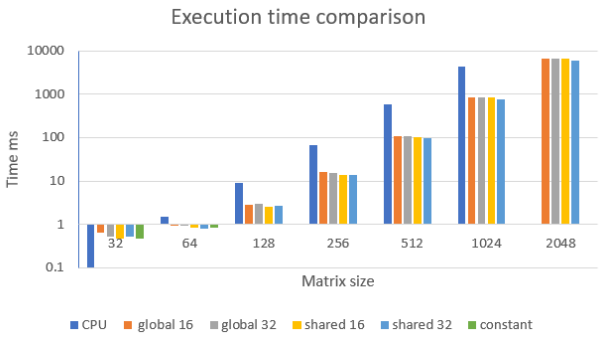


Figure 2: *execution times of using different memory types*

parts of the matrices to the global memory and compared the result of this.
If we compare the global and shared memory, we can see that the implementations with 32 threads is a bit faster than those with 16 threads. The shared memory implementations was always a bit faster than the global memory implementation, but it wasn't by much.



3 Github link

`https://github.com/MaximeRombaut1412/GCA_labs.git`